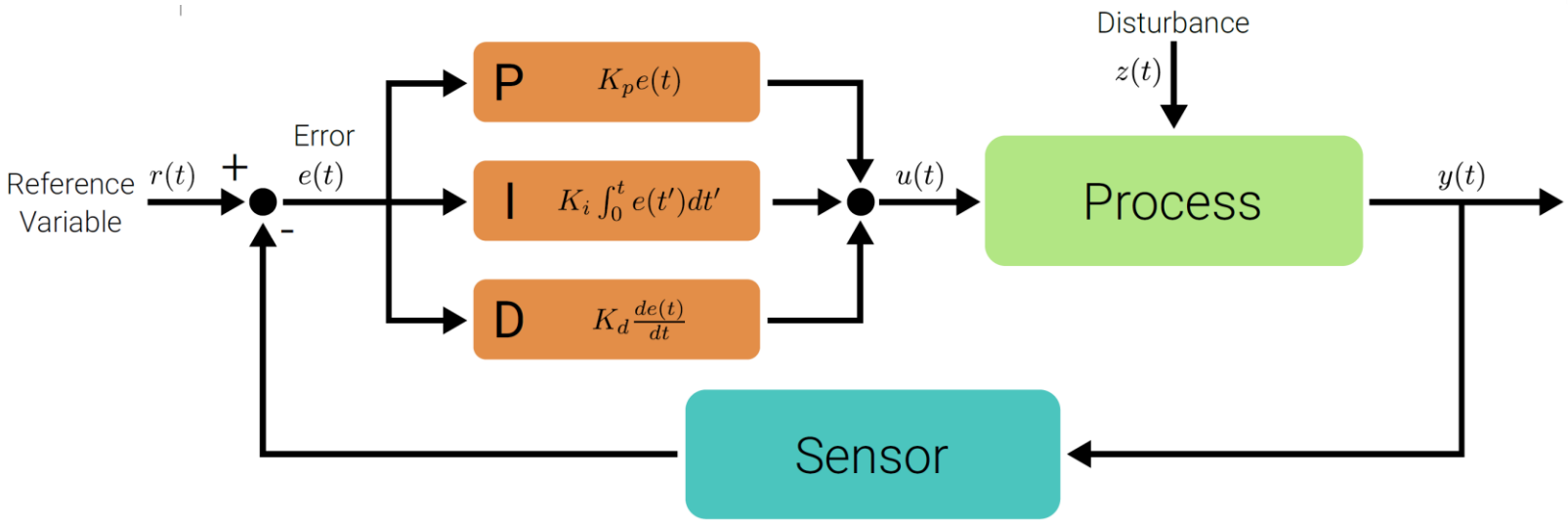
香港城市大學
City University of Hong Kong
Innovating into the Future

CS5493: Tutorial 12
Apollo Control and Tooling

PID Control



PID Control with Continuous Time: $u(t) = Kp[e(t) + \frac{1}{Ti}\int_0^t e(t)dt + Td\frac{de(t)}{dt}]$

PID Control with Discrete Time: $u(k) = K_P e(k) + K_I \sum_{i=0} e(i) + K_D[e(k) - e(k-1)]$

LQR Control

LQR is an **optimal controller** for **linear systems** $\dot{x} = Ax + Bu$

Given a system, it finds the best control action that stabilizes the system *with minimal cost*

For the continuous-time linear system $J = \int_0^\infty \left(\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}\right) dt$

The quadratic cost functional defined as

Q: state cost → penalize how far you are from target

Larger Q: track state more tightly (aggressive)

R: control cost → penalize how much you use the control

Larger R: use control more gently (smooth, conservative)

The optimal control is linear feedback $\mathbf{u} = -K\mathbf{x}$

$$K = R^{-1}B^T P$$

and *P* comes from solving the Algebraic Riccati Equation (ARE)

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$ (solved by libraries)

LQR Control

2-DOF (degree of freedom) Vehicle Lateral Tracking Error Model

$$\boldsymbol{Error_k} = \begin{bmatrix} \text{lateral } \boldsymbol{error} \\ \boldsymbol{lateral\ error\ rate} \\ \boldsymbol{heading\ error} \\ \boldsymbol{heading\ error\ rate} \end{bmatrix}$$

$$\mathrm{Error}_{k+1} = A \cdot Error_k + Bu_k$$

$$\boldsymbol{Error_{k+1}} \quad \begin{bmatrix} \dot{l}_e \\ \ddot{l}_e \\ \dot{\theta}_e \\ \ddot{\theta}_e \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \dfrac{-c_f - c_{r1}}{mv_x} & \dfrac{c_f + c_{r1}}{m} & \dfrac{c_{r1}l_{r1} - c_f l_f}{mv_x} \\ 0 & 0 & 0 & 1 \\ 0 & \dfrac{l_{r1}c_{r1} - l_f c_f}{Iv_x} & \dfrac{l_f c_f - l_{r1}c_{r1}}{I} & \dfrac{-l_{r1}^2 c_{r1} - l_f^2 c_f}{Iv_x} \end{bmatrix} \begin{bmatrix} l_e \\ \dot{l}_e \\ \theta_e \\ \dot{\theta}_e \end{bmatrix} + \begin{bmatrix} 0 \\ \dfrac{c_f}{m} \\ 0 \\ \dfrac{l_f c_f}{I} \end{bmatrix} \delta$$

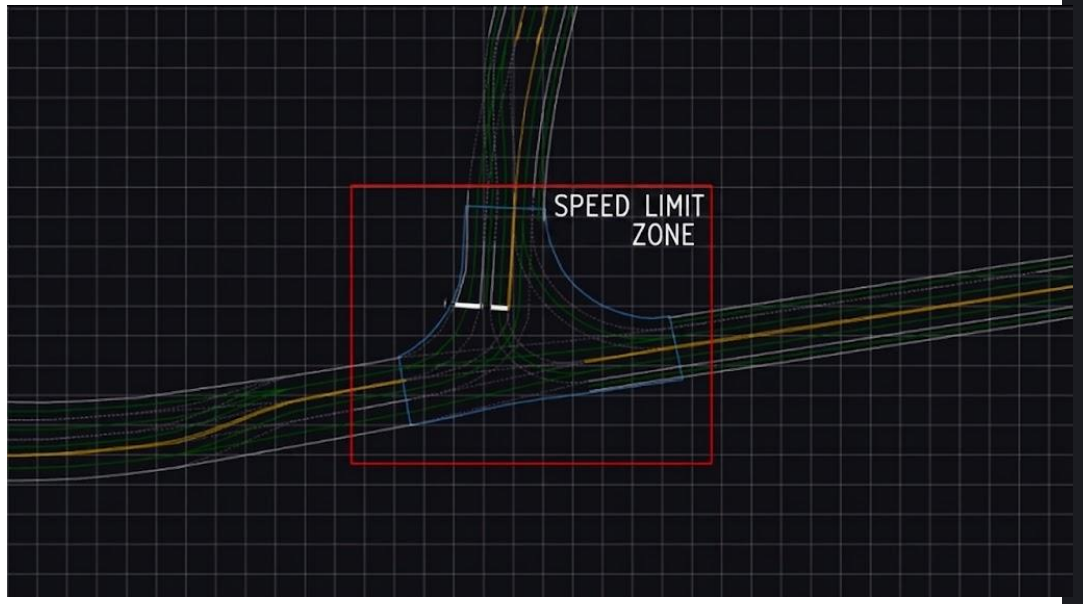
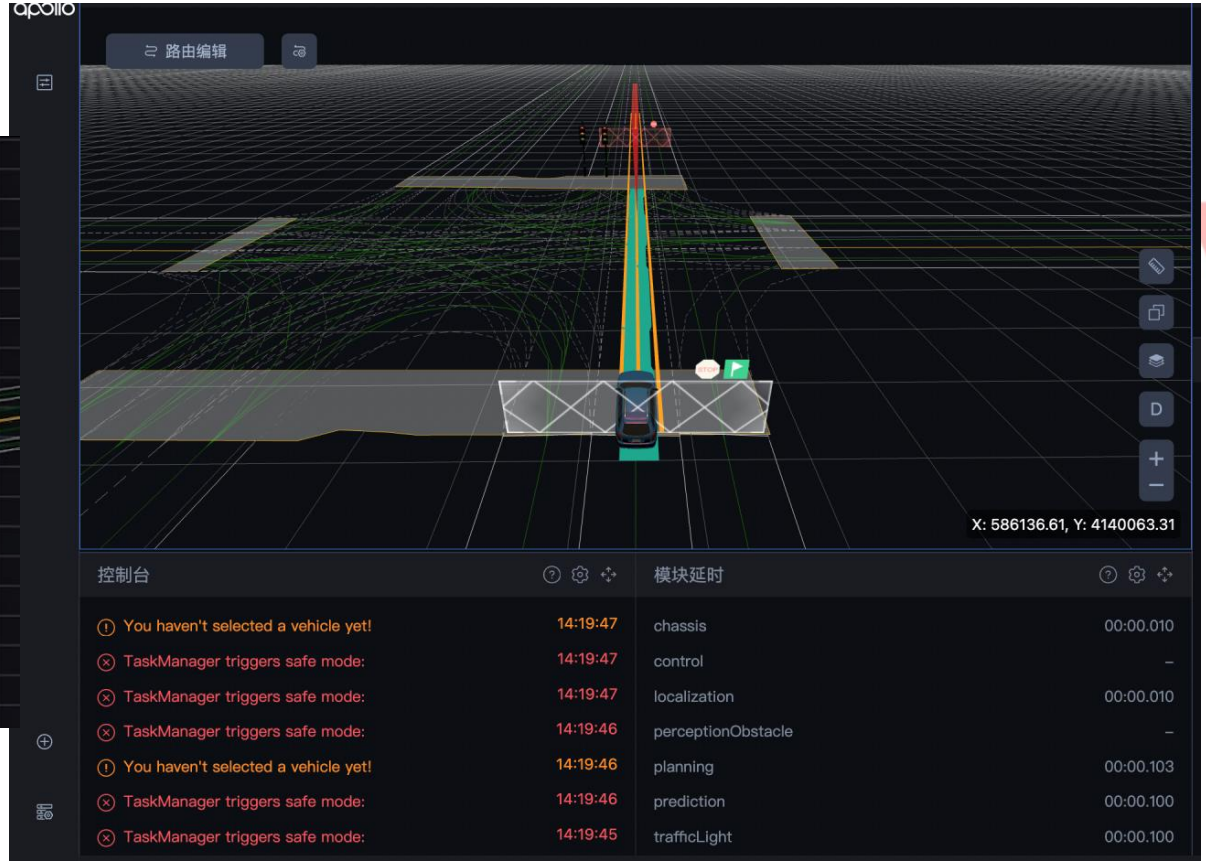
Its cost function

$$J = Error_N^T \cdot Q \cdot Error_N + \sum_{K=0}^{N-1} (Error_K^T \cdot Q \cdot Error_K + u_K^T \cdot R \cdot u_K)$$

LQR + Feedforward Compensation

- The final control output is $\delta_{\text{total}} = \delta_{\text{feedforward}} + \delta_{\text{LQR}} = \delta_f - K \cdot Error_k$
- LQR: computes the control input solely based on the current tracking error state.
  - It does not depend on feedforward information such as path curvature or vehicle speed
  - It only reacts to the magnitude of the current error
- Feedforward Compensation: provides a "baseline steering angle" in advance, so the vehicle has sufficient steering input when entering a curve.
  - The feedforward term does not depend on real-time tracking errors
  - It only uses path geometry and vehicle operating conditions to precompute the steady-state steering demand

$$\delta_f = \frac{mV_x^2}{R(l_f + l_r)}\left[\frac{l_r}{2C_f} - \frac{l_f}{2C_r} + \frac{l_f}{2C_r}k_3\right] + \frac{1}{R}[l_f + l_r - l_r k_3]$$

Apollo Toolings

Traffic rule Plug-in, Lanes and Traffic Light Tools